I tried several approaches to predict the overall OREB for each game. Firstly I wanted to make a baseline model, just using the average OREB overall, in order to judge the other models. First, I uploaded the cleaned dataset. Then, I created a new column pulling the year from the date column that already existed. For my models, since we are predicting future games, I wanted to split training and testing data chronologically. So I found the mean OREB of all the years less than or equal to 2025. I then filtered to find the OREB of the games in 2026. Then I found the mean absolute error between the testing games and the training average. The baseline MAE was 4.590646.

Next, I tried to make a linear regression model. First, I split my data into training and test data again, training on all data before 2026 and testing on 2026 data. I also eliminated the home team, away team, game id, spread, total and date columns. I eliminated the game id, and team columns because I didn't want to overfit the model. I eliminated the date because since I already split the data chronologically I didn't think I really needed it anymore. I eliminated spread and total because I won't know that information when predicting future games. I created the model using all possible predictors so I could determine which ones were significant. The complete linear model had a very low multiple R-squared of 0.01502, and predicted 2026 data with MAE of 4.620572 . This seems pretty bad, given the MAE is higher than my baseline model. The predictors that had p-values below 0.05 were efg_pct_diff and rest_diff. I made a second linear regression model just using these two significant predictors. This model had an even lower multiple R-squared value of 0.0006765, but it also had a slightly lower MAE of 4.574106. This MAE is marginally lower than my baseline model.

Next I tried to make a time series analysis model. I decided to create my time series model month by month. I thought seeing trends over years would probably be too large a span of time to see real effects, but seeing trends day by day would cause problems since there aren't games every day and there would also be way wider variation. First I created a month column using my already existing date column, where I used the floor_date() function to save each game as the first of the month it was in, thus the month column was saving both the year and the month. Then I grouped by the month column, and summarized the data finding the average OREB by month, and I also arranged it chronologically using the month column. I then created a time series object using the monthly average OREB and a frequency of 12. Then, in order to split the data into test and training data, I found the number of rows of data there is in 2026. Then I trained the time series model on the total rows minus the number of rows in 2026. I used the auto_arima function to find the best arima model for the time series analysis, and then I forecasted the predictions. Since the time series is considering changes over months, it can only predict monthly averages as opposed to individual game statistics. I wanted to see how accurate it was on a game by game level though, so I could accurately compare it to my other models. Thus I created a dataset that included my predicted averages by month of the games in 2026. Then I joined that dataset to a dataset with the game level data for the games in 2026. At this point, I calculated the MAE of the real OREBs for those games and the forecasted OREBs from the arima model. This gave me a

MAE of 4.557917 which is technically the best one so far, but is still very similar to both the baseline and the second linear model's MAEs.

Next I tried to create a Neural Net prediction model. Neural Nets need scaled data which is data with values between 0 and 1, so I decided to just use the features of the dataset that already had values between 0 and 1, which were orb_pct_diff, efg_pct_diff, ft_rate_diff, tov_pct_diff, home_back_to_back, away_back_to_back. These were the differentials and the binary variables. I also eliminated all the NA values so they would not interfere. I then had to scale the OREB variable since it was not in between 0 and 1. I did this by finding the overall minimum and maximum OREB values. Then for each individual OREB value I found the distance between it and the minimum over the range from the minimum to the maximum. Then, I split the data between testing and training data chronologically again, also removing the OREB and Date variables so they wouldn't affect the model. I then fit the neuralnet using the neuralnet function from the neuralnet library. I predicted the scaled OREB using the features I had talked about before. I used two hidden layers with 4 and 2 nodes, just because I didn't want the model to take forever to run and I figured I could add more if necessary. I also made sure to set linear.output = true because I am trying to predict a value. Since the model was predicting the scaled OREB, I then had to unscale it by doing the same steps backwards. At that point I could find the MAE of the real OREB and my unscaled predicted OREB. This resulted in the worst MAE so far, of 4.726059.

At this point, I thought I should combine the two models where I had a small amount of success, the time series model and the second linear regression model. I decided I should make another linear model, but where I use the predicted monthly averages from the time series model, and then also the two significant predictors from my second linear regression model. I did the same steps to create my time series model, and then I saved my fitted and forecasted values numerically. Then I saved my fitted values to a dataset with corresponding month and my forecasted values to a dataset with corresponding month and then bound the two datasets together. At that point, I was able to join my bound dataset with my timeseries values to my larger game level dataset. Then, I split my larger dataset chronologically into training and test values yet again, and trained a linear regression using the training values with the time arima predictions for monthly average, the rest_diff and the efg_pct_diff. This model still has a low multiple R squared of 0.02378, but it did result in the lowest MAE so far of 4.534276. Still this is less than .06 different from the baseline MAE which is not very significant.

Overall, I have found that unfortunately, the variables we have created and considered in our cleaned dataset are not very effective at predicting offensive rebounds.

OREB Modeling

2026-03-08

```r
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.5.2

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
library(tidyverse)
```

```
## Warning: package 'tibble' was built under R version 4.5.2

## Warning: package 'tidyr' was built under R version 4.5.2

## Warning: package 'readr' was built under R version 4.5.2

## Warning: package 'purrr' was built under R version 4.5.2

## Warning: package 'lubridate' was built under R version 4.5.2

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v forcats   1.0.0     v readr     2.2.0
## v ggplot2   4.0.0     v stringr   1.6.0
## v lubridate 1.9.5     v tibble    3.3.1
## v purrr     1.2.1     v tidyr     1.3.2

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become error
```

```r
library(neuralnet)
```

```
##
## Attaching package: 'neuralnet'
##
## The following object is masked from 'package:dplyr':
##
##     compute
```

```r
library(forecast)
```

```
## Warning: package 'forecast' was built under R version 4.5.2
```

```
## orb_pct_diff           -0.592649   2.156786  -0.275   0.7835
## ft_rate_diff            1.060035   1.046939   1.013   0.3114
## opp_pts_allowed_diff   -0.005616   0.041976  -0.134   0.8936
## opp_orb_allowed_diff    0.024647   0.159926   0.154   0.8775
## opp_trb_allowed_diff    0.094273   0.075906   1.242   0.2144
## opp_ast_allowed_diff   -0.056750   0.093035  -0.610   0.5419
## opp_tov_forced_diff     0.019775   0.083330   0.237   0.8124
## last5_pts_diff         -0.049401   0.117312  -0.421   0.6737
## last5_pts_allowed_diff  0.028932   0.118097   0.245   0.8065
## last5_oreb_diff         0.031682   0.044629   0.710   0.4778
## last5_off_rtg_diff      1.502157   3.740122   0.402   0.6880
## last5_def_rtg_diff     -1.460027   3.740600  -0.390   0.6963
## last5_net_rtg_diff     -1.434986   3.746798  -0.383   0.7018
## last5_pace_diff         0.006125   0.090389   0.068   0.9460
## rest_diff               0.014033   0.007135   1.967   0.0494 *
## home_back_to_back       0.826009   0.533988   1.547   0.1221
## away_back_to_back       0.763695   0.482753   1.582   0.1138
## year                    0.002775   0.016166   0.172   0.8637
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.73 on 1908 degrees of freedom
##   (2785 observations deleted due to missingness)
## Multiple R-squared:  0.01502,    Adjusted R-squared:  0.003145
## F-statistic: 1.265 on 23 and 1908 DF,  p-value: 0.1793
```

```r
test$predicted_oreb <- predict(linearmodel, newdata = test) #finding predicted values for test data

mae <- mean(abs(test$oreb - test$predicted_oreb), na.rm = TRUE) #calculating MAE
cat("MAE:", mae, "\n")
```

```
## MAE: 4.620572
```

```r
#Linear Model 2

linearmodel2 <- lm(oreb ~ rest_diff + efg_pct_diff, #using predictors that are significant, and that i
                   data = train) #training a model based on all possible predictors

summary(linearmodel2)
```

```
##
## Call:
## lm(formula = oreb ~ rest_diff + efg_pct_diff, data = train)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -15.625  -3.713  -0.549   3.399  62.142
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  21.651793   0.083388 259.650   <2e-16 ***
## rest_diff     0.003591   0.002637   1.362    0.173
## efg_pct_diff -1.054845   0.902606  -1.169    0.243
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
cleandata <- read_csv("Downloads/FINAL_CLEANED_SET - FINAL_CLEANED_SET.csv")
```

```
## Rows: 4995 Columns: 30
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr   (2): home_team, away_team
## dbl  (27): game_id, spread, total, oreb, off_rtg_diff, def_rtg_diff, net_rtg...
## date  (1): date
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
#Baseline Model
cleandata <- cleandata %>% mutate(year = year(ymd(date))) #creating new column for year

# Calculate the average from training seasons
train_mean <- mean(cleandata$oreb[cleandata$year <= 2025], na.rm = TRUE)

test_games <- cleandata %>% filter(year > 2025) #filtering test games as games after 2025

mae_baseline <- mean(abs(test_games$oreb - train_mean), na.rm = TRUE) #finding MAE using season average

cat("Baseline (season average) MAE:", mae_baseline, "\n")
```

```
## Baseline (season average) MAE: 4.590646
```

```r
#Linear Model 1


 #splitting older data into train and new data into test
train <- cleandata %>% filter(year <= 2025) %>% select(-home_team, -away_team, -game_id, -date, -spread
test  <- cleandata %>% filter(year > 2025)  %>% select(-home_team, -away_team, -game_id, -date, -spread

linearmodel <- lm(oreb ~ .,
                  data = train) #training a model based on all possible predictors

summary(linearmodel)
```

```
##
## Call:
## lm(formula = oreb ~ ., data = train)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -15.616  -4.138  -0.278   3.467  26.474
##
## Coefficients: (1 not defined because of singularities)
##                        Estimate Std. Error t value Pr(>|t|)
## (Intercept)           15.758751  32.594652   0.483   0.6288
## off_rtg_diff           1.616488   1.588213   1.018   0.3089
## def_rtg_diff          -1.922477   1.556744  -1.235   0.2170
## net_rtg_diff          -1.744835   1.330040  -1.312   0.1897
## pace_diff                    NA         NA      NA       NA
## efg_pct_diff          -9.442593   4.486102  -2.105   0.0354 *
## tov_pct_diff           7.754034   4.320956   1.795   0.0729 .
```

```r
# Remove rows with NAs in these columns
nn_data <- cleandata %>%
  select(all_of(features), oreb, date, year) %>%
  na.omit()

# Scale oreb
oreb_min <- min(nn_data$oreb) #finding minimum oreb
oreb_max <- max(nn_data$oreb) #finding maximum oreb
nn_data$oreb_scaled <- (nn_data$oreb - oreb_min) / (oreb_max - oreb_min) #finding how far the oreb is a

# Chronological split
train_nn <- nn_data %>% filter(year <= 2025) %>% select(-date, -oreb) # removing oreb and date so they
test_nn  <- nn_data %>% filter(year >  2025) %>% select(-date, -oreb)

# Fit neural net
model_nn <- neuralnet(oreb_scaled ~ orb_pct_diff + efg_pct_diff + ft_rate_diff +
                        #using oreb_scaled as target instead of oreb
                                    tov_pct_diff + home_back_to_back + away_back_to_back,
                      data          = train_nn,
                      hidden        = c(4, 2),  # 2 hidden layers, 4 and 2 nodes
                      linear.output = TRUE)  # makes sure the output isn't between 0 and 1

# Predict then unscale back to real OREB values
pred_scaled <- predict(model_nn, test_nn %>% select(-oreb_scaled))
pred_oreb   <- pred_scaled * (oreb_max - oreb_min) + oreb_min # unscaling using oreb_min and oreb_max

# Unscale actual values too for fair comparison
actual_oreb <- test_nn$oreb_scaled * (oreb_max - oreb_min) + oreb_min

# MAE
mae_nn <- mean(abs(actual_oreb - pred_oreb), na.rm = TRUE)
cat("Neural Net MAE:", mae_nn, "\n")
```

```
## Neural Net MAE: 4.696267
```

```r
#hybrid model using time series
cleandata <- read.csv("Downloads/FINAL_CLEANED_SET - FINAL_CLEANED_SET.csv")
cleandata <- cleandata %>% mutate(year = year(ymd(date)))

monthly_train <- monthly %>% filter(year <= 2025) #training months when year is less than 2025
monthly_test  <- monthly %>% filter(year > 2025) #test months when year is greater than 2025

n_test_ts <- nrow(monthly_test) #finding number of rows where year is greater than 2025
train_ts  <- head(ts_monthly, length(ts_monthly) - n_test_ts) # making ts object on just training rows

model_arima   <- auto.arima(train_ts) #fitting arima model
arima_fitted  <- as.numeric(model_arima$fitted)  # fitted values for training months
arima_forecast <- as.numeric(forecast(model_arima, h = n_test_ts)$mean)  # forecasts for test months

monthly_train$arima_oreb <- arima_fitted #adding values from model to training set
monthly_test$arima_oreb  <- arima_forecast #adding values from model to test set

#making lookup dataset with predictions by month
arima_lookup <- bind_rows(
```

```r
  monthly_train %>% select(month, arima_oreb),
  monthly_test  %>% select(month, arima_oreb)
)

#joining predictions back to original dataset
cleandata2 <- cleandata %>%
  mutate(month = floor_date(ymd(date), "month")) %>%
  left_join(arima_lookup, by = "month")
colnames(cleandata2)
```

```
##  [1] "game_id"                "date"                   "home_team"
##  [4] "away_team"              "spread"                 "total"
##  [7] "oreb"                   "off_rtg_diff"           "def_rtg_diff"
## [10] "net_rtg_diff"           "pace_diff"              "efg_pct_diff"
## [13] "tov_pct_diff"           "orb_pct_diff"           "ft_rate_diff"
## [16] "opp_pts_allowed_diff"   "opp_orb_allowed_diff"   "opp_trb_allowed_diff"
## [19] "opp_ast_allowed_diff"   "opp_tov_forced_diff"    "last5_pts_diff"
## [22] "last5_pts_allowed_diff" "last5_oreb_diff"        "last5_off_rtg_diff"
## [25] "last5_def_rtg_diff"     "last5_net_rtg_diff"     "last5_pace_diff"
## [28] "rest_diff"              "home_back_to_back"      "away_back_to_back"
## [31] "year"                   "month"                  "arima_oreb"
```

```r
#making chronological split of train and test data for my hybrid model
cleandata2 <- cleandata2 %>% mutate(year = year(ymd(date)))
train_hybrid <- cleandata2 %>% filter(year <= 2025)
test_hybrid  <- cleandata2 %>% filter(year > 2025)

#fit hybrid model using arima forecast + best linear features
hybrid_model <- lm(oreb ~ arima_oreb + rest_diff + efg_pct_diff, data = train_hybrid)
summary(hybrid_model)
```

```
##
## Call:
## lm(formula = oreb ~ arima_oreb + rest_diff + efg_pct_diff, data = train_hybrid)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -15.238  -3.927  -0.386   3.514  61.557
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)   6.765895   1.412986   4.788 1.73e-06 ***
## arima_oreb    0.695940   0.065947  10.553  < 2e-16 ***
## rest_diff     0.003100   0.002607   1.189    0.234
## efg_pct_diff -1.326753   0.892578  -1.486    0.137
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.607 on 4706 degrees of freedom
##   (7 observations deleted due to missingness)
## Multiple R-squared:  0.02378,    Adjusted R-squared:  0.02316
## F-statistic: 38.21 on 3 and 4706 DF,  p-value: < 2.2e-16
```

```r
#evaluating and finding MAE
test_hybrid$predicted_oreb <- predict(hybrid_model, newdata = test_hybrid)
```

```
##
## Residual standard error: 5.673 on 4707 degrees of freedom
##   (7 observations deleted due to missingness)
## Multiple R-squared:  0.0006765,  Adjusted R-squared:  0.0002518
## F-statistic: 1.593 on 2 and 4707 DF,  p-value: 0.2034
```

```r
test$predicted_oreb2 <- predict(linearmodel2, newdata = test) #finding predicted values for test data

mae2 <- mean(abs(test$oreb - test$predicted_oreb2), na.rm = TRUE) #calculating MAE
cat("MAE:", mae2, "\n")
```

```
## MAE: 4.574106
```

```r
# Time series analysis - month

monthly <- cleandata %>%
  mutate(month = floor_date(ymd(date), "month")) %>% #creating month column
  group_by(month) %>% #grouping by month
  summarise(avg_oreb = mean(oreb, na.rm = TRUE)) %>% #finding avg oreb per month
  arrange(month) %>%
  mutate(year = year(month))

ts_monthly <- ts(monthly$avg_oreb, frequency = 12) # Convert to ts object

# Train on 2025 and earlier, test on 2026
n_test   <- monthly %>% filter(year > 2025) %>% nrow()  # finding number of rows where year is over 202
train_ts <- head(ts_monthly, length(ts_monthly) - n_test)  # training the model on only the rows not in


model_arima <- auto.arima(train_ts)               # creating time series model, needs ts object
forecasted  <- forecast(model_arima, h = n_test)  # forecasted values based on model, needs a number

# Build lookup table: month -> forecasted value
test_months <- monthly %>% filter(year > 2025) #creating new data frame with only test month
test_months$forecast_oreb <- as.numeric(forecasted$mean) #adding column for predicted monthly means

# Join forecast back to individual games
cleandata <- cleandata %>%
  mutate(month = floor_date(ymd(date), "month")) #creating month column in cleandata

test_games <- cleandata %>%
  filter(year(ymd(date)) > 2025) %>%
  left_join(test_months %>% select(month, forecast_oreb), by = "month")
#joining the forecasted monthly means to larger dataset

# MAE on individual games
mae_ts <- mean(abs(test_games$oreb - test_games$forecast_oreb), na.rm = TRUE)
cat("Time Series MAE (game level):", mae_ts, "\n")
```

```
## Time Series MAE (game level): 4.557917
```

```r
#Neural Net predictions
# Neural nets need scaled data (0 to 1 range), so I included features that have 0 to 1 range
features <- c("orb_pct_diff", "efg_pct_diff", "ft_rate_diff",
              "tov_pct_diff", "home_back_to_back", "away_back_to_back")
```

```r
mae_hybrid <- mean(abs(test_hybrid$oreb - test_hybrid$predicted_oreb), na.rm = TRUE)
cat("Hybrid MAE:", mae_hybrid, "\n")
```

```
## Hybrid MAE: 4.534276
```